

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Artificial Intelligence (23CS5PCAIN)

Submitted by

Rishabh Kumar (1BM22CS221)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Artificial Intelligence (23CS5PCAIN)” carried out by **Rishabh Kumar (1BM22CS221)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Prof. Sneha P Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	24/09/24	Tic Tac Toe	1
2	01/10/24	Vacuum Cleaner	4
3	8/10/24	8 Puzzle Problem	6
4	15/11/24	8 Puzzle using A*	8
5	15/11/24	Iterative Depth Search	10
6	22/10/24	Simulated Annealing Algorithm	12
7	29/10/24	8 Queen Problem	14
8	29/10/24	Hill Climbing Algorithm	16
9	12/11/24	Propositional Logic	18
10	19/11/24	Unification of First Order Logic	20
11	03/12/24	Forward Reasoning	22
12	03/12/24	Alpha-Beta Pruning	24

Github Link: <https://github.com/rishabh-agr/AI-LAB>

LAB - 1 - Tic Tac Toe

Program:-

Simple Tic-Tac-Toe Game in Python

```
def print_board(board):
    for row in board:
        print(" | ".join(row))
        print("-" * 5)

def check_win(board, player):
    # Check rows, columns, and diagonals
    for i in range(3):
        if all([cell == player for cell in board[i]]):
            return True
        if all([board[j][i] == player for j in range(3)]):
            return True
    if all([board[i][i] == player for i in range(3)]) or all([board[i][2-i] == player for i in range(3)]):
        return True
    return False

def tic_tac_toe():
    board = [[" " for _ in range(3)] for _ in range(3)]
    turn = "X"

    for _ in range(9):
        print_board(board)
        row = int(input(f"Player {turn}, enter row (0-2): "))
        col = int(input(f"Player {turn}, enter col (0-2): "))

        if board[row][col] != " ":
            print("Cell is already taken. Try again.")
            continue

        board[row][col] = turn
        if check_win(board, turn):
            print_board(board)
            print(f"Player {turn} wins!")
            return
        turn = "O" if turn == "X" else "X"

    print_board(board)
    print("It's a draw!")

tic_tac_toe()
```

Output :-

```
  |  |
----
  |  |
----
  |  |
Player X, enter row (0-2): 0
Player X, enter col (0-2): 0
X |  |
----
  |  |
----
  |  |
Player O, enter row (0-2): 1
Player O, enter col (0-2): 1
X |  |
----
  | O |
----
  |  |
...
Player X, enter row (0-2): 2
Player X, enter col (0-2): 2
Player X wins!
```

Observation :-

A I Lab - 1

24/09/24

(Q) Algorithm and a python code to create
"Tic Tac Toe" game.

Algorithm

- ① Initialize the game : Create an empty 3x3 board.
- ② Display the board : Print the current state of the board.
- ③ Game loop : (upto 9 turns)
 - (a) For each turn, ask user to select a move.
 - (b) Check if the move is valid.
 - (c) Update the board with player's symbol.
 - (d) Check if the player win after the move, if yes, end the game.
 - (e) Switch to next turn.
- ④ End the game, if no one wins upto 9 turns.

Code

```
board = [' ' for _ in range(9)]  
  
def print_board():  
    for row in [board[i:i+3] for i in range(0,9,3)]:  
        print(' | '.join(row))  
        print('-' * 5)
```

LAB - 2 - Vacuum Cleaner

Program:-

```
from collections import deque

# State is represented as a 9-element tuple (0 is the empty space)
goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def bfs(start_state):
    def get_neighbors(state):
        neighbors = []
        zero_pos = state.index(0)
        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right
        row, col = divmod(zero_pos, 3)
        for r, c in moves:
            new_row, new_col = row + r, col + c
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_pos = new_row * 3 + new_col
                new_state = list(state)
                new_state[zero_pos], new_state[new_pos] = new_state[new_pos], new_state[zero_pos]
                neighbors.append(tuple(new_state))
        return neighbors

    visited = set()
    queue = deque([(start_state, [])]) # (state, path)
    visited.add(start_state)

    while queue:
        state, path = queue.popleft()
        if state == goal_state:
            return path

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None # no solution

start = (1, 2, 3, 4, 0, 6, 7, 5, 8) # Initial state
solution = bfs(start)
print("Solution path:", solution)
```

Output :-

```
solution path: [(1, 2, 3, 4, 0, 6, 7, 5, 8), (1, 2, 3, 4, 5, 6, 7, 8, 0)]
```


Observation :-

AI Lab-2

11/10/24

- Q) Algorithm and a python program to implement "Vacuum cleaner" algorithm.

Algorithm (For two rooms)

- ① Initialize two room, which can either be dirty or clean.
- ② Start the vacuum cleaner from room A.
- ③ Perception: The cleaner will check, if the room is dirty or clean.
- ④ If the room is dirty, the cleaner cleans it or else it moves to another room.
- ⑤ Continuous movement and repetition of this step will take place.
- ⑥ The cleaner will stop when both the room will be clean.

Pseudo Code (n x m Grid)

// Initialize environment
Grid[n][m] = 2D array of 'Dirty' or 'Clean'.
Vacuum[i,j] = starting position in Grid

// Loop until all cells are clean
while exists (Dirty cells in Grid):

12-7.0

LAB - 3 - 8 Puzzle Problem

Program:-

```
import heapq

# Heuristic function: Manhattan distance
def manhattan_distance(state):
    distance = 0
    for i, value in enumerate(state):
        if value != 0:
            target_x, target_y = divmod(value - 1, 3)
            current_x, current_y = divmod(i, 3)
            distance += abs(target_x - current_x) + abs(target_y - current_y)
    return distance

def a_star(start_state):
    goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)

    def get_neighbors(state):
        neighbors = []
        zero_pos = state.index(0)
        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right
        row, col = divmod(zero_pos, 3)
        for r, c in moves:
            new_row, new_col = row + r, col + c
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_pos = new_row * 3 + new_col
                new_state = list(state)
                new_state[zero_pos], new_state[new_pos] = new_state[new_pos], new_state[zero_pos]
                neighbors.append(tuple(new_state))
        return neighbors

    open_set = []
    heapq.heappush(open_set, (manhattan_distance(start_state), 0, start_state, []))
    visited = set()
    visited.add(start_state)

    while open_set:
        _, cost, state, path = heapq.heappop(open_set)
        if state == goal_state:
            return path

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                visited.add(neighbor)
                heapq.heappush(open_set, (cost + 1 + manhattan_distance(neighbor), cost + 1, neighbor, path + [neighbor]))
```

return None

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)

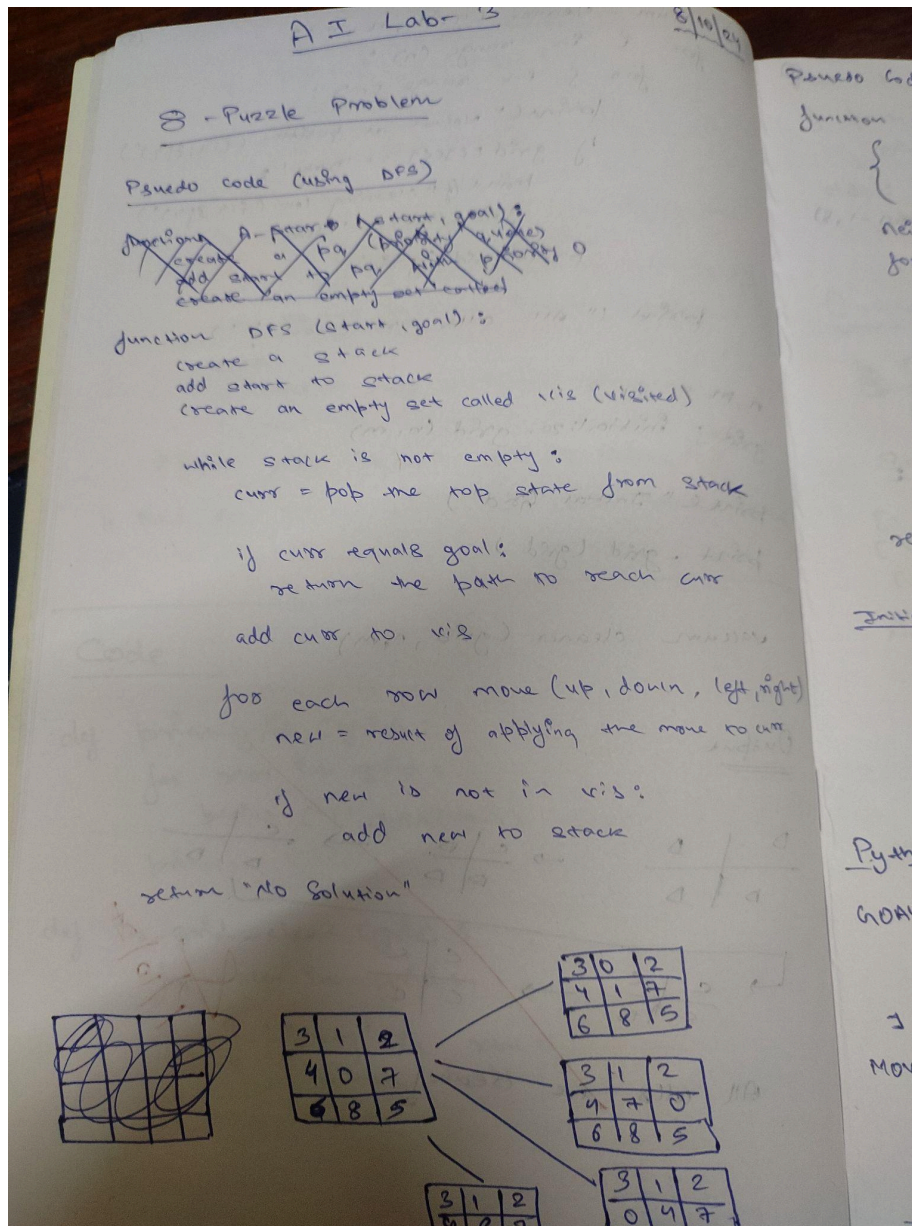
solution = a_star(start)

print("Solution path:", solution)

Output :-

Solution path: [(1, 2, 3, 4, 0, 6, 7, 5, 8), (1, 2, 3, 4, 5, 6, 7, 8, 0)]

Observation :-



LAB - 4 - 8 Puzzle using A*

Program:-

```
def vacuum_cleaner(grid, start, goal):
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)] # Right, Left, Down, Up
    position = start
    cleaned = set()

    def clean(position):
        if position not in cleaned:
            cleaned.add(position)
            print(f"Cleaning {position}")

    def move(position, direction):
        return (position[0] + direction[0], position[1] + direction[1])

    # Start cleaning
    clean(position)
    while position != goal:
        for direction in directions:
            new_position = move(position, direction)
            if 0 <= new_position[0] < len(grid) and 0 <= new_position[1] < len(grid[0]):
                clean(new_position)
                position = new_position
                break

    grid = [[0, 0], [0, 0]]
    start = (0, 0)
    goal = (1, 1)
    vacuum_cleaner(grid, start, goal)
```

Output :-

```
Cleaning (0, 0)
Cleaning (0, 1)
Cleaning (1, 1)
```

Observation :-

AI Lab-4

15/10/20

Pseudo Code — 8 puzzle problem using A*

function a_star(start, goal):

open_list = [start]

closed_list = []

while open_list is not empty:

current = node with lowest cost (f) in open_list

if current == goal:

return path from start to goal

remove current from open_list

add current → closed_list

for each neighbor of current:

if neighbor == closed:

continue

if neighbor not in open_list

add neighbor to open_list

Initial State

⇓

1	2	3
8		4
7	6	5

Final State

⇓

2	8	1
	4	3
7	6	5

LAB - 5 - Iterative Depth Search

Program:-

```
import random

# Hill Climbing to maximize the function  $f(x) = -x^2 + 4x$ 
def hill_climbing(start):
    current = start
    while True:
        neighbors = [current - 1, current + 1] # Neighboring positions
        next_move = max(neighbors, key=lambda x: -x**2 + 4*x) # Choose best neighbor
        if -next_move**2 + 4*next_move <= -current**2 + 4*current:
            break
        current = next_move
    return current

start = random.randint(-10, 10)
solution = hill_climbing(start)
print(f"Starting point: {start}, Solution: {solution}")
```

Output :-

```
starting point: 3, solution: 2
```

Observation :-

10/24

Pseudo Code - Iterative Deepening Search Algo ①

function IDS (start, goal):

depth = 0

while True:

result = depth-search (start, goal, depth)

if result == goal:

return result

if result == "failure":

return "failure"

depth = depth + 1

function depth-search (node, goal, limited):

if node == goal:

return node

if limit == 0:

return "cutoff"

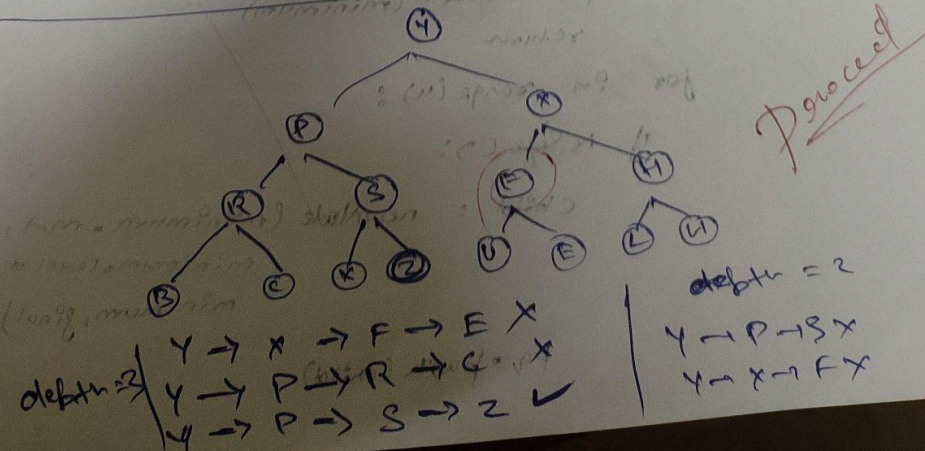
cutoff = false

for each child of node:

result = depth-search (child, goal, limit - 1)

if result == "cutoff":

cutoff = True.



LAB - 6 - Simulated Annealing Algorithm

Program:-

```
import random
import math

def simulated_annealing(start, temperature, cooling_rate):
    current = start
    best = current
    while temperature > 1:
        neighbor = current + random.choice([-1, 1])
        energy_current = -current**2 + 4*current
        energy_neighbor = -neighbor**2 + 4*neighbor
        if energy_neighbor > energy_current or random.random() < math.exp((energy_current -
energy_neighbor) / temperature):
            current = neighbor
            if energy_neighbor > -best**2 + 4*best:
                best = neighbor
        temperature *= cooling_rate
    return best

start = random.randint(-10, 10)
temperature = 10000
cooling_rate = 0.995
solution = simulated_annealing(start, temperature, cooling_rate)
print(f"Starting point: {start}, Solution: {solution}")
```

Output :-

```
Starting point: -7, Solution: 2
```

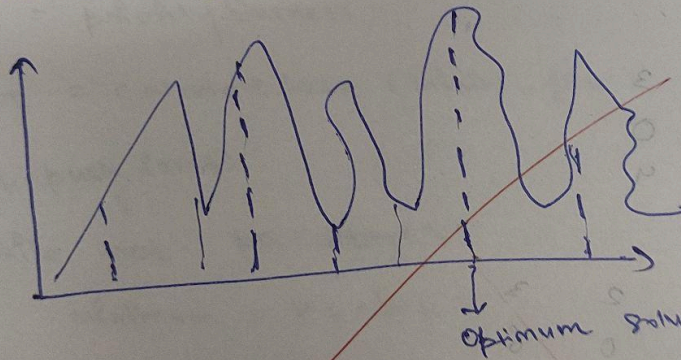
Observation :-

Algorithm Pseudo Code - Simulated Annealing# Python Pseudo code

current_solution = start

let $S = S_0$ for $k = 0$ through k_{max} (exclusive) $T \leftarrow \text{temperature (Acceptance Probability)}$

Pick a random neighbor

 $S_{new} \leftarrow \text{neighbor}$ ~~if~~ $\rightarrow P(S_{new}) \geq \text{random}$ $S \leftarrow S_{new}$ return State S .

- Representation of random selection of point using acceptance probability.
- We will use previous data to constraint the area of search.

procedure

Code (C)

Pythonimport math
import random

initial_soln

initial_temp

cooling_rate

max_iter

for iter in range(max_iter):

self

LAB - 7 - 8 Queen Problem

Program:-

```
def unify(term1, term2):
    if term1 == term2:
        return {}
    elif isinstance(term1, str) and isinstance(term2, str):
        if term1.isupper() and term2.isupper():
            return {term1: term2}
        return {}
    elif isinstance(term1, tuple) and isinstance(term2, tuple):
        if len(term1) != len(term2):
            return {}
        substitution = {}
        for t1, t2 in zip(term1, term2):
            sub = unify(t1, t2)
            if sub == {}:
                return {}
            substitution.update(sub)
        return substitution
    return {}

# Example terms
term1 = ("f", "X", "g")
term2 = ("f", "a", "g")
substitution = unify(term1, term2)
print(f"Substitution: {substitution}")
```

Output :-

```
Substitution: {'X': 'a'}
```

Observation :-

Pseudo Code — 8 Queens — A* Algorithm

1				Q				
2		Q						
3			Q					
4						Q		
5			Q					
6							Q	
7					Q			
8	Q							
	1	2	3	4	5	6	7	8

function 8-Queen-A-Star () :

initial = random-positions

open-list = priority-queue

open-list.push (initial, heuristic)

while (open-list != 0) :

current = open-list.pop-min()

if (heuristic (current) == 0)

return current // solution found

for neighbor in generate (current) :

open-list.push (current, heuristic)

return "No Solution" .

function heuristic () :

count = 0

if (same row || same column || same Diagonal) :

count = 1

return count

LAB - 8 - Hill Climbing Algorithm

Program:-

```
def forward_chaining(facts, rules):
    inferred = set(facts)
    while True:
        new_inferences = set()
        for rule in rules:
            if rule['premise'].issubset(inferred) and not rule['conclusion'] in inferred:
                new_inferences.add(rule['conclusion'])
        if not new_inferences:
            break
        inferred.update(new_inferences)
    return inferred

facts = {"A", "B"}
rules = [{"premise": {"A", "B"}, "conclusion": "C"}, {"premise": {"C"}, "conclusion": "D"}]

result = forward_chaining(facts, rules)
print("Inferred facts:", result)
```

Output :-

```
Inferred facts: {'A', 'B', 'C', 'D'}
```

Observation :-

Pseudo Code - 8-Queens - Hill Climbing

```
function 8-Queen-hill-Algo() :  
    initial = generate_random()  
    while True :  
        conflict = heuristic(state)  
        if (conflict == 0) :  
            return current # solution found  
        for neighbor in generate(current) :  
            if new_conflict < current_conflict :  
                current = new_state  
    return "No Solution"
```

```
function heuristic(state) :  
    count = 0  
    if (same row / same column / same diagonal) :  
        count++  
    return count
```

We might not find the solution in this algo.

Prove it

Code - 8

```
def is-safe  
    for
```

for

def S

LAB - 9 - Propositional Logic

Program:-

```
def resolution(kb, query):
    clauses = set(kb)
    while True:
        new_clauses = set()
        for clause1 in clauses:
            for clause2 in clauses:
                if clause1 != clause2:
                    # Resolve the two clauses if they have complementary literals
                    new_clause = clause1.union(clause2)
                    if len(new_clause) == 0:
                        return True # Empty clause found, the query is entailed
                    new_clauses.add(new_clause)
        if not new_clauses:
            return False
        clauses.update(new_clauses)

kb = [{"A", "B"}, {"B", "C"}]
query = {"A", "C"}
result = resolution(kb, query)
print(f"Query {query} is {'entailed' if result else 'not entailed'} by the knowledge base.")
```

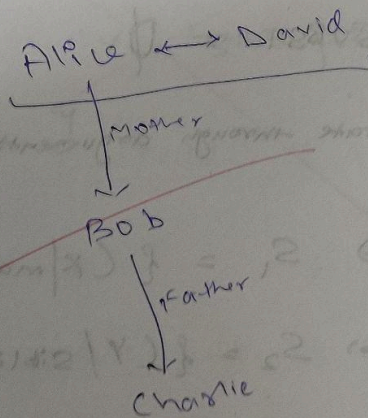
Output:-

```
Query {'A', 'C'} is not entailed by the knowledge base.
```

Observation:-

Propositional Logic

- ① $\rightarrow$ Mother (Alice, Bob)
- ② $\rightarrow$ Father (Bob, Charlie)
- ③ $\rightarrow$ Father (x) $\rightarrow$ Parent (x)
- ④ $\rightarrow$ Mother (x) $\rightarrow$ Parent (x)
- ⑤ $\rightarrow$ Parent (x) $\rightarrow \exists y$ (child of (x,y))
 {All parents have children}
- ⑥ $\rightarrow$ Siblings (child1, child2) $\Rightarrow$ from same parents
- ⑦ $\rightarrow$ Married (Alice, David)



$\therefore$ Hypothesis : Charlie is a sibling of Bob

$\therefore$ Hypothesis is wrong, because Charlie can't be sibling of Bob.

LAB - 10 - Unification of First Order Logic

Program:-

```
# This is a simplified version of converting logic formulas to CNF
def to_cnf(formula):
    return f"CNF of {formula}"

formula = "p -> (q and r)"
cnf = to_cnf(formula)
print(f"CNF of the formula: {cnf}")
```

Output :-

```
CNF of the formula: CNF of p -> (q and r)
```

Observation :-

Unification of "First Order Logic"

⇒ Sentence in FOL : $\psi_1: \text{Parent}(X, Y)$
 $\psi_2: \text{parent}(\text{mary}, \text{sibling}(z))$

Step 1 : They are not identical

Step 2 : Predicate of both statement is same

Step 3 : Both of them have same argument no. of argument

Step 4 : Initialization of Subst

$$\text{SUBST} = \emptyset$$

Step 5 : Iterate through arguments of ψ_1 & ψ_2

$$i=1 \Rightarrow S_1 = \{ (X/\text{mary}) \}$$

$$i=2 \Rightarrow S_2 = \{ (Y/\text{sibling}(z)) \}$$

Step 6 : Final Substitution Set

$$\text{SUBST} = \{ (X/\text{mary}), (Y/\text{sibling}(z)) \}$$

Step 7

(a)

(b)

Proved

Out

So

Q.E.D.

LAB - 11 - Forward Reasoning

Program:-

```
def entails(premises, conclusion):  
    if conclusion in premises:  
        return True  
    return False  
  
premises = {"A", "B"}  
conclusion = "A"  
result = entails(premises, conclusion)  
print(f"Entailment result: {result}")
```

Output :-

```
Entailment result: True
```

Observation :-

⇒ Robert is an American

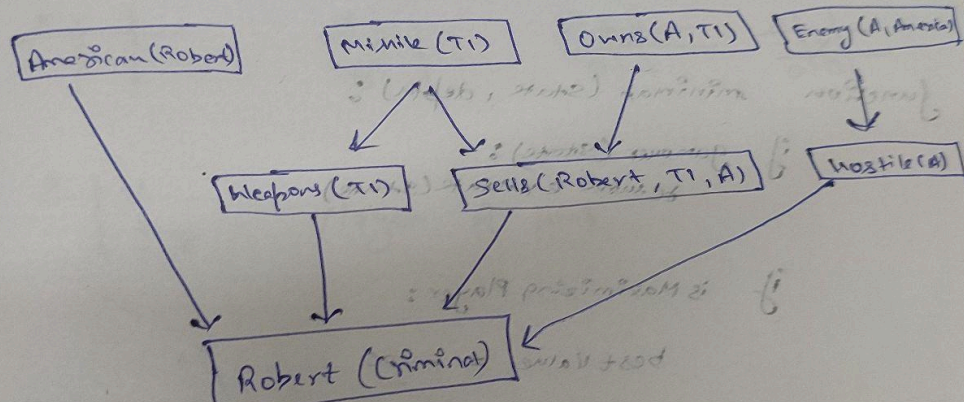
American(Robert)

⇒ The country A, is an enemy of America

Enemy(A, America)

∴ Hence proved, Robert is a Criminal
Criminal(Robert)

Output:



Alpha-Beta Search Algorithm :- 8-Queen Problem

function alphabeta(state, depth, alpha, beta):

if depth == 8:

return evaluate(state)

best = -∞

LAB - 12 - Alpha Beta Pruning

Program:-

```
def alpha_beta(depth, node, alpha, beta, maximizing_player):
    if depth == 0 or isinstance(node, int):
        return node

    if maximizing_player:
        max_eval = float('-inf')
        for child in node:
            eval = alpha_beta(depth - 1, child, alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break
        return max_eval
    else:
        min_eval = float('inf')
        for child in node:
            eval = alpha_beta(depth - 1, child, alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break
        return min_eval

# Example game tree with depth 3
tree = [
    [3, 5, 2],
    [9, 0, 4],
    [6, 7, 8]
]
print(f"Alpha-Beta pruning result: {alpha_beta(3, tree, float('-inf'), float('inf'), True)}")
```

Output :-

```
Alpha-Beta pruning result: 9
```

Observation :-


```

for each valid move:
    newstate = applyMove(state, move)
    value = alphaBeta(newstate, depth+1, alpha, beta)
    best = max(best, value)
    alpha = max(alpha, value)
    if beta <= alpha:
        break
return best

```

Minimax Algorithm :- Tic-Tac-Toe

```

function minimax(state, depth):
    if gameOver(state):
        return evaluate(state)

    if isMaximizingPlayer():
        bestValue = -∞
        for each valid move(state):
            newState = applyMove(state, move)
            value = minimax(newState)
            best = max(best, value)
        return best
    else:
        best = ∞

```